

Smart Construction Connect

AI-Powered Construction Matchmaking Platform — FYP Documentation (Final v4.0)

Environment: Localhost only | **Data Layer:** JSON files | **Backend:** FastAPI (Python) |
Frontend: React.js (Vite) + Tailwind CSS

Table of Contents

1. [Project Overview](#)
2. [Problem Statement](#)
3. [Vision & Goals](#)
4. [User Personas](#)
5. [User Roles & Permissions](#)
6. [Client Experience](#)
7. [Company Dashboard](#)
8. [Supplier Dashboard](#)
9. [Admin Dashboard](#)
10. [AI Matchmaking Engine](#)
11. [Notification System](#)
12. [Review & Trust System](#)
13. [Full-Stack Architecture](#)
14. [FastAPI Backend](#)
15. [Frontend Architecture](#)
16. [Page Inventory \(MVP\)](#)
17. [JSON Data Schema](#)
18. [Data Flow](#)
19. [UI Theme System \(Light / Dark Mode\)](#)
20. [Development Phases](#)
21. [Testing Strategy](#)
22. [UI/UX Design System](#)
23. [Out of Scope](#)
24. [Feature Comparison Matrix](#)
25. [Key Differentiators](#)
26. [Risk Register](#)

1. Project Overview

Smart Construction Connect is an AI-powered, full-stack web platform that intelligently connects four key stakeholders in the construction industry. The application runs entirely on **localhost** and uses a **FastAPI Python backend** to handle all CRUD operations, with data persisted in structured **JSON files**.

Stakeholder	Role
 Client	Homeowners or buyers looking to build or renovate
 Construction Company	Service providers seeking verified project leads
 Material Supplier	Vendors selling construction materials
 Admin	Platform guardian ensuring trust and quality

The platform replaces fragmented, manual processes — word-of-mouth referrals, cold calls, physical visits — with a centralized, intelligent, and transparent matchmaking system powered by AI.

Tech Stack

Layer	Technology
Frontend	React.js (Vite), Tailwind CSS, Framer Motion
State Management	Zustand
Routing	React Router v6
Backend	FastAPI (Python 3.11+)
API Communication	Axios (frontend) ↔ FastAPI REST endpoints
Data Layer	Local JSON files (read/written by FastAPI)
AI Chat UI	Custom UI + Claude / OpenAI API (Phase 3)
Icons	Lucide React
Theme	Light / Dark mode with localStorage persistence

2. Problem Statement

The construction industry in Pakistan (and developing markets broadly) faces:

- No centralized directory of verified, rated construction companies
- No price transparency — clients are often overcharged
- Supplier discovery is manual — companies don't know where to source materials efficiently
- No accountability — bad actors (fraudulent companies, fake reviews) go unchecked
- Language barrier — most platforms are English-only, excluding Urdu-speaking clients

Smart Construction Connect directly addresses all five of these problems.

Key Statistics (Pakistan Context)

- 60%+ of construction disputes arise from mismatched expectations
 - Clients typically contact 5-10 companies manually before deciding
 - Material procurement is largely offline, causing 20-30% cost inefficiencies
 - No existing platform serves this space in Pakistan at scale
-

3. Vision & Goals

Vision

To become the most trusted AI-driven construction ecosystem in Pakistan — where clients find verified companies, companies get quality leads, and suppliers grow their business efficiently.

FYP Goals

- Build a complete, role-based **React frontend** with realistic JSON dummy data
- Implement a **FastAPI backend** to handle all CRUD operations on JSON files
- Demonstrate a working AI chatbot UI for premium client matchmaking
- Design a scalable system architecture that can evolve into a real product
- Showcase admin governance tools for platform trust
- Support **Light Mode** and **Dark Mode** UI themes

Long-Term Goals (Post-FYP)

- Real database (PostgreSQL) replacing JSON files
- Payments integration (Stripe / JazzCash / EasyPaisa)
- Mobile app (React Native)

- Cloud deployment (Vercel frontend + Railway / AWS backend)
 - Real-time chat with WebSockets
-

4. User Personas

Persona 1: Raza (Client — Normal Mode)

- **Age:** 38, Government employee
- **Goal:** Build a 10-Marla house in DHA Lahore on a 1.5 Crore budget
- **Pain:** Doesn't know which company is genuine; has been misquoted before
- **Tech literacy:** Medium — uses WhatsApp, browses Facebook
- **Needs:** Simple filtering, company ratings, compare 2-3 companies side-by-side

Persona 2: Hania (Client — Premium Mode)

- **Age:** 29, IT professional
- **Goal:** Custom contemporary home; wants AI assistance in shortlisting
- **Pain:** Too busy to manually research; wants smart recommendations
- **Tech literacy:** High — comfortable with AI chatbots
- **Needs:** AI assistant in English/Urdu, explainable recommendations, fast decision-making

Persona 3: M. Shahid (Construction Company Owner)

- **Age:** 45, runs a mid-size company
- **Goal:** Get 3-4 new verified project leads per month
- **Pain:** Marketing is word-of-mouth; no digital portfolio
- **Needs:** Professional profile page, lead management, material sourcing

Persona 4: Tariq (Material Supplier)

- **Age:** 52, brick and cement wholesale dealer
- **Goal:** Connect directly with construction companies; reduce idle inventory
- **Pain:** Depends on brokers; no digital storefront
- **Needs:** Product listings, inventory management, order tracking

Persona 5: Admin

- **Role:** Platform operator
- **Goal:** Maintain platform integrity and trust
- **Needs:** Full control over approvals, bans, content moderation, analytics

5. User Roles & Permissions

Feature	Client (Normal)	Client (Premium)	Company	Supplier	Admin
Browse Companies	✓	✓	✗	✗	✓
Browse Materials	✓	✓	✓	✓	✓
Browse Suppliers	✓	✓	✓	✓	✓
Filter & Search	✓	✓	✓	✓	✓
Compare Companies	✓ (manual)	✓	✗	✗	✗
AI Chat Assistant	✗	✓	✗	✗	✗
AI Recommendations	✗	✓	✗	✗	✗
Request Quotes	✓	✓	✗	✗	✗
Manage Projects	✗	✗	✓	✗	✗
Add / Edit Materials	✗	✗	✗	✓	✓
Post Materials	✗	✗	✗	✓	✗
Update Ratings	✗	✗	✗	✓	✓
Write Reviews	✓	✓	✗	✗	✗
Approve Companies	✗	✗	✗	✗	✓
Ban Users	✗	✗	✗	✗	✓
View Audit Logs	✗	✗	✗	✗	✓
Moderate Reviews	✗	✗	✗	✗	✓
Toggle Light/Dark Mode	✓	✓	✓	✓	✓

Note on Onboarding: Companies and Suppliers enter a **"Pending"** state after signup. They see a read-only dashboard and cannot operate until Admin approves them. This state is managed via a `status` flag in JSON files and enforced by FastAPI.

6. Client Experience

6.1 Normal Mode

Flow: Dashboard → Browse Companies → Filter → Compare → Request Quote

Features:

- Search and browse verified construction companies
- Browse materials and suppliers with ratings and pricing
- Filter by: budget range, rating, location (city/area), specialization, availability, verified-only toggle
- Sort by: Best Match, Highest Rated, Lowest Price, Most Reviews
- Side-by-side comparison of up to 3 companies (price range, rating, project types, location, completion time, verified status)
- Request a quote: fill Budget, Area, Timeline, Description → company notified
- View company profile: portfolio, reviews, team size, past projects
- Save/bookmark companies to a wishlist
- Toggle between Light Mode 🌞 and Dark Mode 🌙

6.2 Premium Mode — AI Matchmaking

Flow: Activate Premium → AI Chat (Sara/Marcus) → Requirements Collected → Live Analysis Panel → Top 5 Recommendations

Split-Screen UI Layout:

AI CHAT (LEFT)

Sara: "Assalam o Alaikum!
Let's find your perfect
construction company.
What is your budget?"

User: "1.5 crore"

Sara: "Great! What area
are you building on?"

[Text input] [Send]

[🎤 Voice] [اردو]

LIVE PANEL (RIGHT)

📄 Requirements Summary

Budget: 1.5 Crore

Area: 10 Marla

Style: Modern

Location: DHA Lahore

Timeline: 6 months

⚙️ AI Processing Logs

✓ Budget analyzed

✓ Location matched

⌚ Calculating style score...

🏆 Top Recommendations

1. BuildPro Lahore94%

✓

2. UrbanCraft Co.88%

3. Noor Builders82%

AI Agent Personas (user selects on first launch):

- **Sara** — Female, friendly, warm tone (default)
- **Marcus** — Male, professional, formal tone

Data Sara/Marcus Collects:

Field	Example
Budget	"1.5 crore", "50 lakh"
Area	"10 marla", "1 kanal"
Location	"DHA Lahore", "Bahria Town Karachi"
Construction Style	"Modern", "Traditional", "Mixed"
Timeline	"6 months", "as soon as possible"
Special Requirements	"basement needed", "solar-ready"

Recommendation Card (per result):

- Company name + verified badge
- Match score (e.g., 94%)

- Score breakdown: Price, Location, Rating, Style, Availability
- Why it matched: *"Budget fits / DHA experience / 4.8★rating"*
- CTA buttons: [View Profile] [Request Quote] [Compare]

7. Company Dashboard

Sections

Section	Description
 Incoming Requests	All client quote requests with status (New / Viewed / Responded / Accepted / Declined)
 Active Projects	Track ongoing projects with milestones and client info
 Material Marketplace	Browse and order from verified suppliers; filter by category/price/rating
 Profile Management	Edit portfolio, team, specializations, upload project photos
 Reviews	View and respond to client reviews
 Analytics	Leads received, response rate, profile views

Project Status Flow



Project Milestones



Incoming Request Card

Client: Ahmed Raza

Budget: 1.2 - 1.5 Crore

Area: 10 Marla | Location: DHA Lahore

Timeline: 6 months

Message: "Looking for modern design..."

| [Accept] [Decline] [Ask for Details] |

8. Supplier Dashboard

Sections

Section	Description
 Product / Material Listings	Add, edit, delete materials with name, description, pricing, rating, availability
 Inventory Tracker	Monitor stock levels with low-stock alerts
 Order Management	View and respond to purchase inquiries from companies
 Pricing Manager	Set bulk-pricing tiers, discount rules
 Rating Management	View current rating; upgrade/update based on reviews and performance
 Sales Analytics	Order volume, top-selling materials, revenue trends

Material Object (JSON — managed via FastAPI)

```
json
{
  "id": "mat_001",
  "supplierId": "sup_001",
  "name": "OPC Cement 50kg",
  "description": "High-strength Ordinary Portland Cement, ideal for structural use."
  "category": "Cement",
  "pricePerUnit": 1050,
  "unit": "bag",
  "stock": 500,
  "minOrderQty": 10,
  "brand": "Lucky Cement",
  "rating": 4.6,
  "deliveryAvailable": true,
  "location": "Lahore",
  "isActive": true
}
```

Supplier CRUD Actions (via FastAPI)

Action	HTTP Method	Endpoint
View all my materials	GET	/api/suppliers/{id}/materials
Add new material	POST	/api/materials
Update material	PUT	/api/materials/{id}
Delete material	DELETE	/api/materials/{id}
Update supplier profile	PUT	/api/suppliers/{id}
Update supplier rating	PATCH	/api/suppliers/{id}/rating

9. Admin Dashboard

9.1 Overview Analytics

Total Users	Companies	Suppliers	Requests
1,240	87	43	312
+12% ↑	Pending: 5	Pending: 2	Active: 94

9.2 User Management

- Paginated table: ID, Name, Email, Role, Status, Joined Date, Last Active
- Actions: View Profile, Suspend, Ban, Restore, Change Role
- Status badges: Active | Suspended | Banned | Pending
- Search by name or email; filter by role, status, date range

9.3 Company Verification Panel

Flow:

Submitted Application
↓
Admin Reviews Documents (License, NTN, Portfolio)
↓
Approve → Verified Badge Added → Company Notified
Reject → Reason Required → Company Notified

9.4 Supplier Verification Panel

Same flow as Company Verification, with supplier-specific documents (business registration, product catalog). Includes product listing review before approval.

9.5 Content Moderation

- Flagged reviews appear in a queue (user-reported)
- Admin actions: Approve, Remove, Escalate
- Fake/spam material listings: Remove with reason

9.6 Audit Logs

```
[2025-06-10 14:32] ADMIN:  Approved company "BuildPro Lahore"  
[2025-06-10 14:29] SYSTEM: New company signup – "Noor Builders"  
[2025-06-10 14:15] CLIENT: Raza Ahmed submitted quote request #QR-2041  
[2025-06-10 14:10] AI:      Premium session started – User #U-882  
[2025-06-10 14:05] SUPPLIER: Tariq added material "OPC Cement 50kg"
```

Filterable by: User, Action Type, Date Range | Exportable as CSV (simulated for FYP)

9.7 Material & Supplier Oversight

Task	Admin Action
Review pending supplier	Approve / Reject with reason
Remove fake material listing	Delete via FastAPI + log action
Update supplier rating manually	PATCH <code>/api/suppliers/{id}/rating</code>
View all materials across platform	GET <code>/api/materials</code>

10. AI Matchmaking Engine

10.1 Input Schema (JSON)

json

```
{
  "budget": 15000000,
  "area_marla": 10,
  "location": { "city": "Lahore", "area": "DHA Phase 5" },
  "style": "modern",
  "timeline_months": 6,
  "special_requirements": ["basement", "solar-ready"]
}
```

10.2 Scoring Algorithm

```
Final Score = Weighted Sum of:
price_match_score   × 0.35
location_score      × 0.25
rating_score        × 0.20
style_similarity     × 0.10
availability_score  × 0.10
```

Factor	Calculation
Price Match (35%)	$\boxed{1.0}$ if budget in range; penalized linearly if below; $\boxed{0.9}$ if budget exceeds
Location (25%)	Same city + same area = 1.0 / Same city only = 0.7 / Different city = 0.3
Rating (20%)	$\boxed{\text{company.avg_rating} / 5.0}$ — only companies with ≥ 3 reviews
Style (10%)	Keyword/tag matching: Modern↔Contemporary = 0.85, Modern↔Traditional = 0.30
Availability (10%)	0 active projects = 1.0 / $\leq 2 = 0.8$ / $\leq 5 = 0.5$ / $> 5 = 0.2$

10.3 RAG Pipeline (Phase 3)

- **Vector Store:** ChromaDB (local) → Pinecone (production)
- **Embeddings:** OpenAI text-embedding-3-small or local Ollama model
- **Indexed documents:** Company profiles, project descriptions, reviews, material listings
- **LLM for explanation:** Claude API (natural language explanation generation)

10.4 Mock AI (Phase 1 & 2)

- Hardcoded company data with metadata in JSON files
- Scoring algorithm runs on frontend with dummy weights

- Fake "processing logs" with `setTimeout` delays for UX realism
- Bilingual responses (Urdu/English) pre-written for common question patterns

11. Notification System

All notifications are simulated in-app using Zustand state — no email/SMS/push for FYP.

Trigger	Recipient	Message
New quote request	Company	"Ahmed Raza requested a quote for 10 Marla house"
Quote responded	Client	"Bright Builders responded to your request"
Application approved	Company/Supplier	"Your account has been verified ✓"
Application rejected	Company/Supplier	"Your application was rejected. Reason: [X]"
New review received	Company	"You received a 5★ review from a client"
Low stock alert	Supplier	"OPC Cement stock below 50 bags"
Material added	Admin	"Supplier Tariq added a new material listing"

12. Review & Trust System

Review Flow

```
Project Completed
↓
Client Receives Review Prompt
↓
Client Rates: Overall, Communication, Quality, Timeline
↓
Review Published (after spam check)
↓
Company Can Respond Publicly (once only)
```

Trust Signals on Company Profile

-  Verified Badge (admin-approved)
-  Aggregate Rating (weighted; recent reviews weighted more)
-  Completed Projects Count

- 🕒 Member Since
- 💬 Response Rate %

Anti-Fraud Rules

- Reviews only from clients who completed a project with that company
 - One review per project
 - Admin can flag/remove reviews with notes
 - Spam filter for keyword stuffing (simulated for FYP)
-

13. Full-Stack Architecture



```
|   ├── requests.json
|   ├── projects.json
|   ├── reviews.json
|   ├── notifications.json
|   ├── ai_sessions.json
|   └── admin_logs.json
```

CORS Configuration

FastAPI must enable CORS for the React dev server:

```
python

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

14. FastAPI Backend

Project Structure

```
backend/
├── main.py                # FastAPI app entry point
├── requirements.txt       # Python dependencies
├── routers/
│   ├── users.py
│   ├── companies.py
│   ├── suppliers.py
│   ├── materials.py      ← NEW
│   ├── products.py
│   ├── requests.py
│   ├── projects.py
│   ├── reviews.py
│   ├── notifications.py
│   └── admin.py
├── models/
│   ├── user.py           # Pydantic models
│   └── company.py
```

```
|   ├── supplier.py
|   ├── material.py           ← NEW
|   └── ...
└── utils/
    ├── json_handler.py      # Helpers: read_json(), write_json()
    └── data/
        ├── users.json
        ├── companies.json
        ├── suppliers.json
        ├── materials.json   ← NEW
        └── ...
```

requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
pydantic==2.7.1
python-multipart==0.0.9
```

JSON Handler Utility (utils/json_handler.py)

```
python

import json
from pathlib import Path

DATA_DIR = Path(__file__).parent.parent / "data"

def read_json(filename: str) -> list:
    path = DATA_DIR / filename
    if not path.exists():
        return []
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

def write_json(filename: str, data: list) -> None:
    path = DATA_DIR / filename
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)
```

Material Pydantic Model (models/material.py)

```
python
```



```
from pydantic import BaseModel
from typing import Optional

class MaterialBase(BaseModel):
    supplierId: str
    name: str
    description: str
    category: str
    pricePerUnit: float
    unit: str
    stock: int
    minOrderQty: int
    brand: Optional[str] = None
    rating: float = 0.0
    deliveryAvailable: bool = True
    location: str
    isActive: bool = True

class MaterialCreate(MaterialBase):
    pass

class MaterialUpdate(BaseModel):
    name: Optional[str] = None
    description: Optional[str] = None
    pricePerUnit: Optional[float] = None
    stock: Optional[int] = None
    rating: Optional[float] = None
    isActive: Optional[bool] = None

class Material(MaterialBase):
    id: str
```

Materials Router (`routers/materials.py`)

python

```
from fastapi import APIRouter, HTTPException
from models.material import Material, MaterialCreate, MaterialUpdate
from utils.json_handler import read_json, write_json
import uuid

router = APIRouter(prefix="/api/materials", tags=["Materials"])

@router.get("/", response_model=list[Material])
def get_all_materials():
    return read_json("materials.json")

@router.get("/{material_id}", response_model=Material)
def get_material(material_id: str):
    materials = read_json("materials.json")
    mat = next((m for m in materials if m["id"] == material_id), None)
    if not mat:
        raise HTTPException(status_code=404, detail="Material not found")
    return mat

@router.post("/", response_model=Material, status_code=201)
def create_material(payload: MaterialCreate):
    materials = read_json("materials.json")
    new_material = {"id": f"mat_{uuid.uuid4().hex[:6]}", **payload.model_dump()}
    materials.append(new_material)
    write_json("materials.json", materials)
    return new_material

@router.put("/{material_id}", response_model=Material)
def update_material(material_id: str, payload: MaterialUpdate):
    materials = read_json("materials.json")
    for i, mat in enumerate(materials):
        if mat["id"] == material_id:
            updated = {**mat, **payload.model_dump(exclude_none=True)}
            materials[i] = updated
            write_json("materials.json", materials)
            return updated
    raise HTTPException(status_code=404, detail="Material not found")

@router.delete("/{material_id}")
def delete_material(material_id: str):
    materials = read_json("materials.json")
    filtered = [m for m in materials if m["id"] != material_id]
    if len(filtered) == len(materials):
        raise HTTPException(status_code=404, detail="Material not found")
```

```
write_json("materials.json", filtered)
return {"message": "Material deleted"}
```

Supplier Rating Endpoint (`routers/suppliers.py` — excerpt)

python

```
@router.patch("/{supplier_id}/rating")
def update_supplier_rating(supplier_id: str, rating: float):
    if not (0.0 <= rating <= 5.0):
        raise HTTPException(status_code=400, detail="Rating must be between 0.0 and 5.0")
    suppliers = read_json("suppliers.json")
    for i, sup in enumerate(suppliers):
        if sup["id"] == supplier_id:
            suppliers[i]["rating"] = round(rating, 1)
            write_json("suppliers.json", suppliers)
            return {"message": "Rating updated", "rating": rating}
    raise HTTPException(status_code=404, detail="Supplier not found")
```

Main App Entry Point (`main.py`)

python

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routers import users, companies, suppliers, materials, products, requests, projects

app = FastAPI(title="Smart Construction Connect API", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(users.router)
app.include_router(companies.router)
app.include_router(suppliers.router)
app.include_router(materials.router)
app.include_router(products.router)
app.include_router(requests.router)
app.include_router(projects.router)
app.include_router(reviews.router)
app.include_router(notifications.router)
app.include_router(admin.router)

@app.get("/")
def root():
    return {"message": "Smart Construction Connect API is running."}
```

Running the Backend

```
bash

# Install dependencies
pip install -r requirements.txt

# Start the server
uvicorn main:app --reload --port 8000

# Interactive API docs
http://localhost:8000/docs
```

Full API Endpoint Reference

Resource	Method	Endpoint	Description
Users	GET	/api/users	List all users
Users	POST	/api/users	Create user
Users	PUT	/api/users/{id}	Update user
Users	DELETE	/api/users/{id}	Delete user
Companies	GET	/api/companies	List all companies
Companies	POST	/api/companies	Register company
Companies	PUT	/api/companies/{id}	Update company
Companies	PATCH	/api/companies/{id}/verify	Approve/reject company
Suppliers	GET	/api/suppliers	List all suppliers
Suppliers	POST	/api/suppliers	Register supplier
Suppliers	PUT	/api/suppliers/{id}	Update supplier profile
Suppliers	PATCH	/api/suppliers/{id}/rating	Update supplier rating
Materials	GET	/api/materials	List all materials
Materials	GET	/api/materials/{id}	Get single material
Materials	POST	/api/materials	Add new material
Materials	PUT	/api/materials/{id}	Update material
Materials	DELETE	/api/materials/{id}	Delete material
Requests	GET	/api/requests	List quote requests
Requests	POST	/api/requests	Submit quote request
Requests	PATCH	/api/requests/{id}/status	Update request status
Projects	GET	/api/projects	List projects
Projects	POST	/api/projects	Create project
Projects	PATCH	/api/projects/{id}/milestone	Advance milestone

Resource	Method	Endpoint	Description
Reviews	GET	<code>/api/reviews</code>	List reviews
Reviews	POST	<code>/api/reviews</code>	Submit review
Reviews	PATCH	<code>/api/reviews/{id}/flag</code>	Flag review
Notifications	GET	<code>/api/notifications/{userId}</code>	Get user notifications
Notifications	PATCH	<code>/api/notifications/{id}/read</code>	Mark as read
Admin	GET	<code>/api/admin/logs</code>	Get audit logs
Admin	POST	<code>/api/admin/logs</code>	Append log entry

15. Frontend Architecture

Folder Structure

```
frontend/  
src/  
├── components/  
│   ├── common/           # Button, Card, Badge, Modal, Input, Avatar, Toast, ThemeToggle  
│   ├── layout/           # Navbar, Sidebar, Footer  
│   ├── client/           # ClientDashboard, AIChat, CompanyCard, CompareTable  
│   ├── company/          # CompanyDashboard, RequestCard, ProjectCard, Milestones  
│   ├── supplier/         # SupplierDashboard, MaterialCard, InventoryTable, RatingBadge  
│   └── admin/            # AdminDashboard, UserTable, VerificationPanel, LogStream  
├── pages/  
│   ├── Landing.jsx  
│   ├── Login.jsx  
│   ├── Signup.jsx  
│   ├── client/  
│   ├── company/  
│   ├── supplier/  
│   └── admin/  
├── store/  
│   ├── authStore.js  
│   ├── notificationStore.js  
│   ├── themeStore.js      ← NEW  
│   └── materialStore.js    ← NEW  
└── hooks/
```

```
|   |─ useMaterials.js      ← NEW (Axios calls to FastAPI)
|   |─ useTheme.js         ← NEW
|─ api/
|   |─ axios.js            ← Axios instance (baseUrl: localhost:8000)
|   |─ materialsApi.js     ← NEW
|   |─ suppliersApi.js     ← NEW
|   |─ companiesApi.js
|─ utils/                  # Helpers, formatters, scoring algorithm
|─ styles/                 # Tailwind config, global CSS (light/dark vars)
```

Axios Instance (**api/axios.js**)

javascript

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8000',
  headers: { 'Content-Type': 'application/json' },
});

export default api;
```

Materials API Service (**api/materialsApi.js**)

javascript

```
import api from './axios';

export const getAllMaterials = () => api.get('/api/materials');
export const getMaterial = (id) => api.get(`/api/materials/${id}`);
export const createMaterial = (data) => api.post('/api/materials', data);
export const updateMaterial = (id, data) => api.put(`/api/materials/${id}`, data);
export const deleteMaterial = (id) => api.delete(`/api/materials/${id}`);
```

Theme Store (**store/themeStore.js**)

javascript

```
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

const useThemeStore = create(
  persist(
    (set) => ({
      theme: 'light',
      toggleTheme: () =>
        set((state) => ({ theme: state.theme === 'light' ? 'dark' : 'light' })),
    }),
    { name: 'scc-theme' }
  )
);

export default useThemeStore;
```

Reusable Components

Component	Usage
<ThemeToggle>	☀️ / 🌙 toggle button in Navbar — persisted via localStorage
<StatusBadge>	Pending / Approved / Rejected / Banned
<MatchScoreRing>	Circular progress showing AI match %
<VerifiedBadge>	Gold checkmark on company/supplier profile
<MaterialCard>	Browse, search, and filter materials
<RatingStars>	Interactive or display-only star rating for materials/suppliers
<LogStream>	AI analysis logs panel (scrollable)
<CompanyCard>	Browse, compare, and recommendation panel
<RequestCard>	Company dashboard incoming requests
<Skeleton>	Loading states for all card types

16. Page Inventory (MVP)

Shared Pages

Page	Path
Landing Page	/
Login (mock)	/login
Signup (mock)	/signup
Browse Materials	/materials
Browse Suppliers	/suppliers
404 Not Found	*

Client Pages

Page	Path
Dashboard	/client/dashboard
Browse Companies	/client/companies
Company Detail	/client/companies/:id
Compare Companies	/client/compare
AI Chat (Premium)	/client/ai-chat
Premium Upgrade	/client/upgrade
My Requests	/client/requests
Saved Companies	/client/saved
My Reviews	/client/reviews
Profile Settings	/client/settings

Company Pages

Page	Path
Dashboard	/company/dashboard

Page	Path
Incoming Requests	/company/requests
Active Projects	/company/projects
Project Detail	/company/projects/:id
Material Marketplace	/company/marketplace
Public Profile Builder	/company/profile
Settings	/company/settings

Supplier Pages

Page	Path
Dashboard	/supplier/dashboard
Material Management	/supplier/materials
Add / Edit Material	/supplier/materials/new, /supplier/materials/:id/edit
Inventory	/supplier/inventory
Orders	/supplier/orders
Rating & Profile	/supplier/profile
Settings	/supplier/settings

Admin Pages

Page	Path
Dashboard	/admin/dashboard
User Management	/admin/users
Company Verification	/admin/verify/companies
Supplier Verification	/admin/verify/suppliers
Material Oversight	/admin/materials
Content Moderation	/admin/moderation

Page	Path
Activity Logs	/admin/logs
Analytics	/admin/analytics

17. JSON Data Schema

┆ All JSON files live in `backend/data/`. FastAPI reads and writes them on every API call.

`materials.json` ← NEW

json

```
[
  {
    "id": "mat_001",
    "supplierId": "sup_001",
    "name": "OPC Cement 50kg",
    "description": "High-strength Ordinary Portland Cement, ideal for structural use",
    "category": "Cement",
    "pricePerUnit": 1050,
    "unit": "bag",
    "stock": 500,
    "minOrderQty": 10,
    "brand": "Lucky Cement",
    "rating": 4.6,
    "deliveryAvailable": true,
    "location": "Lahore",
    "isActive": true
  },
  {
    "id": "mat_002",
    "supplierId": "sup_002",
    "name": "Red Clay Bricks (Class A)",
    "description": "Kiln-fired, weather-resistant Class A bricks for load-bearing wa",
    "category": "Bricks",
    "pricePerUnit": 14,
    "unit": "piece",
    "stock": 20000,
    "minOrderQty": 1000,
    "brand": "Punjab Bricks Co.",
    "rating": 4.3,
    "deliveryAvailable": false,
    "location": "Sheikhupura",
    "isActive": true
  }
]
```

suppliers.json (updated)

json

```
[
  {
    "id": "sup_001",
    "userId": "usr_020",
    "businessName": "Tariq Cement Depot",
    "location": "Lahore",
    "categories": ["Cement", "Bricks"],
    "rating": 4.5,
    "verified": true,
    "status": "approved",
    "createdAt": "2025-01-15T08:00:00Z"
  }
]
```

users.json

json

```
[
  {
    "id": "usr_001",
    "name": "Ahmed Raza",
    "email": "ahmed@example.com",
    "role": "client",
    "isPremium": false,
    "isVerified": true,
    "status": "active",
    "createdAt": "2025-01-10T08:00:00Z"
  },
  {
    "id": "usr_002",
    "name": "Hania Khan",
    "email": "hania@example.com",
    "role": "client",
    "isPremium": true,
    "isVerified": true,
    "status": "active",
    "createdAt": "2025-02-05T10:00:00Z"
  }
]
```

companies.json

json

```
[
  {
    "id": "comp_001",
    "userId": "usr_010",
    "name": "Bright Builders",
    "city": "Lahore",
    "area": "DHA",
    "bio": "Premium construction specialists with 15+ years in DHA.",
    "specialization": ["Modern", "Commercial"],
    "minPrice": 100000000,
    "maxPrice": 200000000,
    "rating": 4.8,
    "reviewCount": 32,
    "projectsCompleted": 47,
    "activeProjects": 2,
    "verified": true,
    "status": "approved",
    "teamSize": 45,
    "yearsInBusiness": 12,
    "portfolioImages": ["proj1.jpg", "proj2.jpg"]
  }
]
```

requests.json

json

```
[
  {
    "id": "req_001",
    "clientId": "usr_001",
    "companyId": "comp_001",
    "budget": 150000000,
    "area": 10,
    "location": "DHA Lahore",
    "style": "Modern",
    "timeline": 6,
    "message": "Looking for a 10 marla modern house with basement.",
    "status": "pending",
    "createdAt": "2025-06-10T09:00:00Z"
  }
]
```

projects.json

json

```
[
  {
    "id": "proj_001",
    "companyId": "comp_001",
    "clientId": "usr_001",
    "requestId": "req_001",
    "title": "10 Marla Modern House - DHA Phase 5",
    "status": "in_progress",
    "currentMilestone": "Structure",
    "milestones": ["Foundation", "Structure", "Roofing", "Finishing", "Handover"],
    "startDate": "2025-03-01",
    "estimatedEndDate": "2025-09-01"
  }
]
```

reviews.json

json

```
[
  {
    "id": "rev_001",
    "projectId": "proj_001",
    "clientId": "usr_001",
    "companyId": "comp_001",
    "ratingOverall": 5,
    "ratingQuality": 5,
    "ratingCommunication": 4,
    "ratingTimeline": 5,
    "body": "Excellent work, very professional team.",
    "isFlagged": false,
    "isVisible": true,
    "createdAt": "2025-09-05T12:00:00Z"
  }
]
```

notifications.json

json

```
[
  {
    "id": "notif_001",
    "userId": "usr_010",
    "type": "new_request",
    "message": "Ahmed Raza requested a quote for 10 Marla house",
    "read": false,
    "createdAt": "2025-06-10T09:05:00Z"
  }
]
```

admin_logs.json

json

```
[
  {
    "id": "log_001",
    "adminId": "usr_admin",
    "actionType": "company_approved",
    "targetId": "comp_001",
    "targetType": "company",
    "reason": "All documents verified",
    "createdAt": "2025-06-10T14:32:00Z"
  }
]
```

18. Data Flow

The full request lifecycle from user action to JSON update:


```
1. User interacts with frontend (e.g., Supplier adds a new material)
   ↓
2. React component calls materialsApi.createMaterial(data)
   ↓
3. Axios sends POST /api/materials to FastAPI (localhost:8000)
   ↓
4. FastAPI validates payload via Pydantic model (MaterialCreate)
   ↓
5. FastAPI reads materials.json via read_json()
   ↓
6. New material appended with generated ID
   ↓
7. write_json() saves updated array back to materials.json
   ↓
8. FastAPI returns 201 Created with new material object
   ↓
9. Axios receives response; React updates Zustand materialStore
   ↓
10. UI re-renders with new material visible in supplier dashboard
```

19. UI Theme System (Light / Dark Mode)

Implementation Strategy

The theme is driven by a `data-theme` attribute on `<html>` combined with CSS custom properties. Zustand persists the user's preference to `localStorage`.

Tailwind Configuration (`tailwind.config.js`)

```
javascript

module.exports = {
  darkMode: 'class', // Toggle by adding 'dark' class to <html>
  content: ['./src/**/*.{js,jsx}'],
  theme: { extend: {} },
  plugins: [],
};
```

Theme Toggle Component

```
jsx
```

```
import useThemeStore from '../store/themeStore';
import { Sun, Moon } from 'lucide-react';

export default function ThemeToggle() {
  const { theme, toggleTheme } = useThemeStore();

  // Apply dark class to document root
  document.documentElement.classList.toggle('dark', theme === 'dark');

  return (
    <button
      onClick={toggleTheme}
      className="p-2 rounded-full bg-gray-100 dark:bg-gray-800 transition-colors"
      aria-label="Toggle theme"
    >
      {theme === 'light' ? <Moon size={18} /> : <Sun size={18} />}
    </button>
  );
}
```

Color Tokens (Light / Dark)

Token	Light Mode	Dark Mode
Background	#F8FAFC	#0F172A
Surface (cards)	#FFFFFF	#1E293B
Border	#E2E8F0	#334155
Text Primary	#111827	#F1F5F9
Text Secondary	#6B7280	#94A3B8
Primary Accent	#1D4ED8	#3B82F6
Success	#22C55E	#4ADE80
Warning	#EAB308	#FDE047
Danger	#EF4444	#F87171

20. Development Phases

Phase 1 — Static UI (Weeks 1-3)

Goal: All pages look great, navigation works, JSON dummy data renders via FastAPI.

- Landing page with hero, features, how-it-works sections
- Login / Signup forms with role selection
- Client Normal Mode: Browse + Filter + Compare (companies + materials + suppliers)
- Client Premium Mode: Split-screen AI chat UI + dummy recommendations
- Company Dashboard with dummy requests and projects
- Supplier Dashboard with material listings (CRUD forms wired to FastAPI)
- Admin Dashboard with tables and status badges
- Light/Dark mode toggle wired across all pages
- Responsive layout (mobile, tablet, desktop)

Deliverable: Fully navigable React app connected to FastAPI with JSON persistence.

Phase 2 — Routing, State & Full CRUD (Weeks 4-5)

Goal: App behaves like a real product with full backend integration.

- React Router with role-based protected routes
- Zustand stores (auth, requests, notifications, materials, theme)
- All CRUD operations wired to FastAPI endpoints via Axios
- Request flow: client submits → FastAPI writes `requests.json` → appears in company dashboard
- Admin actions: approve/reject → FastAPI updates `companies.json`
- Supplier adds/edits/deletes material → FastAPI updates `materials.json`
- Notification state simulation
- AI chat state machine (multi-step requirement collection)

Deliverable: Working multi-role app with full data persistence via FastAPI + JSON.

Phase 3 — AI Integration (Weeks 6-9)

Goal: Real AI chat integration (Claude / OpenAI API).

- Real AI chat with Claude API / OpenAI API
- RAG pipeline for matchmaking (LangChain + ChromaDB local)
- Scoring algorithm fully implemented in frontend utilities
- File uploads for company docs (handled client-side, stored as base64 in JSON)
- FastAPI `/api/ai/match` endpoint as optional proxy for scoring

Deliverable: AI-powered matchmaking working on localhost.

Phase 4 — Polish & Demo (Weeks 10-11)

Goal: Demo-ready FYP submission.

- Responsive polish on all screens
- Loading states, error states, empty states
- Framer Motion animations (page transitions, card hover, AI log stream)
- Comprehensive Light/Dark mode testing across all components
- Presentation slides and demo video
- Documentation finalization + README
- FastAPI auto-generated docs (`/docs`) as bonus demo

Deliverable: Submission-ready FYP.

21. Testing Strategy

Backend Tests (pytest)

```
bash
```

```
pip install pytest httpx  
pytest tests/
```

```
python
```

```

# tests/test_materials.py
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_get_materials():
    response = client.get("/api/materials")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_create_material():
    payload = {
        "supplierId": "sup_001",
        "name": "Test Steel",
        "description": "Test",
        "category": "Steel",
        "pricePerUnit": 500,
        "unit": "kg",
        "stock": 100,
        "minOrderQty": 10,
        "rating": 4.0,
        "deliveryAvailable": True,
        "location": "Lahore",
        "isActive": True
    }
    response = client.post("/api/materials", json=payload)
    assert response.status_code == 201
    assert response.json()["name"] == "Test Steel"

```

Frontend Tests (Jest + React Testing Library)

- Individual components (MaterialCard, ThemeToggle, RatingStars)
- Zustand store actions
- API service mocks (Axios)

Integration Tests (Cypress)

- Login → role redirect flow
- Supplier adds material → appears in company marketplace
- Admin approves company → verified badge appears
- Theme toggle → persists on page refresh

Manual Functional Testing Checklist

Scenario	Expected Result
Supplier adds material via form	POST to FastAPI; <code>materials.json</code> updated; card appears in UI
Supplier edits material price	PUT to FastAPI; <code>materials.json</code> updated; new price shown
Supplier deletes material	DELETE to FastAPI; material removed from JSON and UI
Admin updates supplier rating	PATCH to FastAPI; rating updated in <code>suppliers.json</code>
User toggles Dark Mode	<code><html></code> gets <code>dark</code> class; preference saved to localStorage
Client browses materials	GET from FastAPI; all active materials displayed
Company views marketplace	GET <code>/api/materials</code> ; filtered by category
Admin approves company	PATCH to FastAPI; <code>companies.json</code> updated; verified badge added

22. UI/UX Design System

Design Principles

1. **Clarity first** — Users should never be confused about where they are or what to do next
2. **Role-aware UI** — Each role has a distinct color accent and layout identity
3. **Theme-aware** — Every component must support both Light and Dark mode
4. **Mobile-responsive** — All layouts work on 375px and above
5. **Progressive disclosure** — Show complexity only when needed

Color Palette

Token	Hex	Usage
<code>--primary</code>	 <code>#1D4ED8</code>	Buttons, links, highlights
<code>--primary-dark</code>	 <code>#1E3A8A</code>	Hover states
<code>--client-accent</code>	 <code>#10B981</code>	Client UI accents (Emerald)
<code>--company-accent</code>	 <code>#4F46E5</code>	Company UI accents (Indigo)
<code>--supplier-accent</code>	 <code>#F59E0B</code>	Supplier UI accents (Amber)
<code>--admin-accent</code>	 <code>#DC2626</code>	Admin UI accents (Red)

Token	Hex	Usage
--premium	 #FBBF24	Premium badge, gold indicators
--success	 #22C55E	Verified badges, success states
--danger	 #EF4444	Errors, bans, rejections
--warning	 #EAB308	Pending states, alerts

Typography

Style	Font	Size	Weight
Display / Page Title	Inter	36px	700
Section Heading	Inter	28px	600
Card Title	Inter	18px	600
Body	Inter	16px	400
Small / Caption	Inter	13px	400
Label	Inter	12px	500

Animations (Framer Motion)

Trigger	Animation
Page transition	Fade + slide up (0.3s)
Card hover	Scale 1.02 + shadow increase
AI log entry	Slide in from left (staggered)
Recommendation card	Fade + scale in (staggered delay)
Modal open	Scale from 0.9 + fade
Score bar	Width animate from 0% on enter
Theme toggle	Rotate + scale icon (0.2s)

Responsive Breakpoints

Breakpoint	Width	Layout
Mobile	< 640px	Single column; collapsed sidebar
Tablet	640-1024px	2-column grid; slide-out sidebar
Desktop	> 1024px	Full layout; persistent sidebar

23. Out of Scope

Feature	Reason
Payment Processing	Requires PCI compliance, banking integration
Cloud Deployment	FYP runs on localhost only
Mobile App (iOS/Android)	Separate future project
Real-time Chat (WebSockets)	Post-FYP with Socket.io
Email / SMS Notifications	Simulated in-app only
Third-party Map Integration	Cost; mockable for FYP
Multi-language CMS	Content is hardcoded for FYP
Legal Contracts / Digital Signatures	Out of scope
Real Database (PostgreSQL)	JSON files used instead for FYP
JWT Authentication	Mock login with Zustand; role set via form

24. Feature Comparison Matrix

Feature	Normal Client	Premium Client	Company	Supplier	Admin
Browse Companies	✓	✓	—	—	✓
Browse Materials	✓	✓	✓	✓	✓
Browse Suppliers	✓	✓	✓	✓	✓

Feature	Normal Client	Premium Client	Company	Supplier	Admin
Filter & Compare	✓	✓	✓	✓	✓
Request Quote	✓	✓	—	—	—
AI Chat (Sara)	✗	✓	—	—	—
AI Recommendations	✗	✓	—	—	—
View Reviews	✓	✓	✓	—	✓
Write Review	✓ (post-project)	✓	—	—	—
Manage Projects	—	—	✓	—	—
Marketplace Access	—	—	✓	—	—
Add / Edit Materials	—	—	—	✓	✓
Order Management	—	—	—	✓	—
Update Rating	—	—	—	✓ (own)	✓ (all)
Approve Users	—	—	—	—	✓
View All Logs	—	—	—	—	✓
Ban / Suspend Users	—	—	—	—	✓
Platform Analytics	—	—	—	—	✓
Toggle Light/Dark	✓	✓	✓	✓	✓

25. Key Differentiators

Aspect	Regular Directory	Smart Construction Connect
Discovery	Manual search	AI-powered matchmaking
Language	English only	Urdu + English
Trust	No verification	Admin-verified + reviews
Matching	Generic	Score-based with explanation
Supplier link	Not present	Integrated material marketplace

Aspect	Regular Directory	Smart Construction Connect
Governance	None	Full admin control layer
Data Persistence	Static	FastAPI CRUD → JSON files
Theme	Light only	Light + Dark mode
Market	Generic	Pakistan-specific (PKR, Marla, DHA)

"The only platform in Pakistan where a homeowner can describe their dream house in Urdu, and receive AI-matched, verified, scored construction companies in under 60 seconds."

26. Risk Register

Risk	Likelihood	Impact	Mitigation
AI API rate limits in demo	Medium	High	Pre-generate responses for demo scenarios
LLM gives wrong recommendations	Medium	Medium	Scoring algorithm overrides LLM; show breakdown
FastAPI JSON race condition (concurrent writes)	Low	Medium	FYP is single-user localhost; add file locks post-FYP
Mock data looks unrealistic	Low	Medium	Use real Pakistani company names and Lahore locations
Phase 3 AI not complete for demo	Medium	High	Ensure Phase 1-2 are demo-ready as fallback
Urdu AI responses broken	Low	High	Test all Urdu inputs; hardcode fallback responses
Admin panel complexity causes delays	Low	Medium	Build admin last; core flows take priority
Dark mode inconsistencies across components	Medium	Low	Use centralized CSS token system; test all screens

27. Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — AI technique that retrieves relevant documents before generating a response
Match Score	A weighted percentage indicating how well a company fits a client's requirements
Verified Badge	Visual indicator that a company/supplier has been approved by Admin
Premium Mode	AI-powered client mode with chatbot and recommendation features
Marla	Pakistani unit of area; 1 Marla $\approx$ 25.3 m ²
NTN	National Tax Number (Pakistani business registration)
RBAC	Role-Based Access Control — restricts system access based on user role
CRUD	Create, Read, Update, Delete — the four basic data operations
FastAPI	A modern, fast Python web framework for building APIs
Pydantic	Python data validation library used by FastAPI for request/response models
Axios	Promise-based HTTP client for the browser, used in the React frontend
MOQ	Minimum Order Quantity — minimum units a supplier will sell
DHA	Defence Housing Authority — premium residential area in Pakistani cities
Light/Dark Mode	UI theme setting toggled by the user; persisted in localStorage

Document Version: 4.0 (Full-Stack FYP Edition) | Backend: FastAPI | Frontend: React (Vite) | Data: JSON | Last Updated: 2025 Project: Smart Construction Connect | Author: Smart Construction Connect FYP Team